



Projet Ingénierie de réseaux

Prédiction du trafic

Parcours A

Réalisé par :

DEHBI Yassine
AIT RAMI Mohamed
ASSOU Ayoub
BENNIS Kawtar
SAMRAN Fatima Zohra

2020-2021

Table des matières

1	Introduction	2
2	Etat de l'art	2
2.1	Neural Network	2
2.2	Support Vector Machine	3
2.3	ARIMA	3
2.4	Comparaison des algorithmes	3
3	Demos et Résultats	4
3.1	LSTM	4
3.1.1	Implémentation	5
3.1.2	Echelle de prédiction	5
3.1.3	Remarques	5
3.2	ARIMA	7
3.2.1	Commentaires	8
3.3	SVM	8
3.3.1	RBF Kernel	8
3.3.2	Polynomial Kernel	8
4	Conclusion	8

1 Introduction

Etant donné un réseau satellite, nous désirons prédire le débit agrégé sur la voie aller ou sur la voie retour de ce réseau. Pour faire cela, nous disposons d'algorithmes de Machine learning, qui en général permettent de construire un modèle prédictif à partir des données historiques et de l'utiliser pour prédire les nouvelles données. L'objectif de ce projet est de faire une comparaison de trois méthodes de prédiction en termes de complexité et de précision pour voir si une prédiction fiable du débit peut être faite. Pour faire cela, nous allons utiliser nos algorithmes sur différents scénarios pour mieux voir la différence.

2 Etat de l'art

2.1 Neural Network

Ici, on est face à une série temporelle, qu'on doit prédire sa valeur à partir des réalisations précédentes. Les réseaux de neurones traditionnels ne permettent pas aux informations de persister. Les réseaux de neurones récurrents répondent à ce problème. Se comportant en boucle. Les informations peuvent donc persister. Malheureusement, en pratique les RNN ne semblent pas capables d'apprendre les dépendances "à long terme". Le problème a été exploré par Hochreiter[1]Bengio, et al.[2]. Les LSTM(Long Short Term Memory) sont explicitement conçus pour éviter le problème de dépendance à long terme, grâce à une mémoire interne. Cette cellule mémoire permet de maintenir un état aussi longtemps que nécessaire.

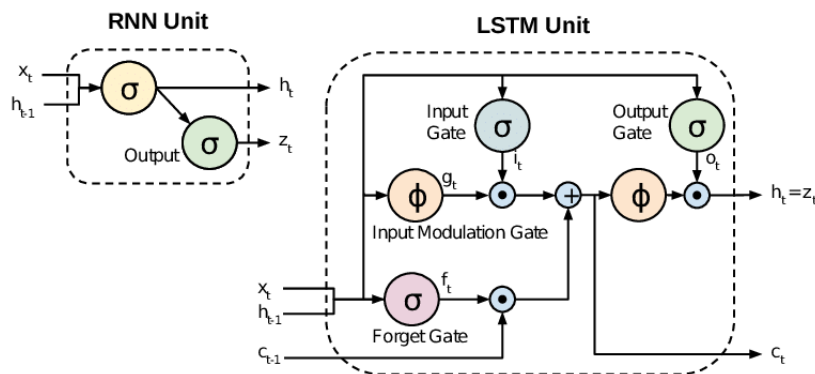


Figure 1: Cellule LSTM

- Input Gate : décide si l'entrée doit modifier le contenu de la cellule
- Forget Gate : décide s'il faut remettre à 0 le contenu de la cellule
- Output Gate : décide si le contenu de la cellule va influencer sur la sortie du neurone

2.2 Support Vector Machine

Les SVMs sont une famille d'algorithmes d'apprentissage automatique qui permettent de résoudre des problèmes tant de classification que de régression ou de détection d'anomalie. Ils sont connus pour leurs solides garanties théoriques, leur grande flexibilité ainsi que leur simplicité d'utilisation. Comme le montre la Figure, leur principe est simple : il ont pour but de séparer les données en classes à l'aide d'une frontière aussi « simple » que possible, de telle façon que la distance entre les différents groupes de données et la frontière qui les sépare soit maximale. Cette distance est aussi appelée « margin ».

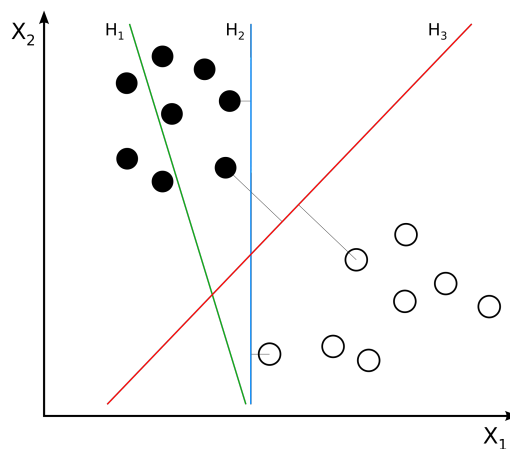


Figure 2: SVM

SVR permet de résoudre les problèmes de la régression non linéaire. C'est le cas de notre série temporelle. La régression des vecteurs de support (SVR) utilise les mêmes principes que la SVM pour la classification, avec seulement quelques différences mineures. Tout d'abord, parce que la sortie est un nombre réel, il devient très difficile de prédire les informations disponibles, qui ont des possibilités infinies.

2.3 ARIMA

ARIMA fait partie des algorithmes de prédiction statistiques. Elle est basée sur trois modèles mathématiques: Autoregressive (AR), Integrate (I), MA (Moyenne mobiles). La combinaison de ces trois modèles se fait par des coefficients ou poids. Un modèle ARIMA est donc un triplet (p,r,q) . L'objectif est de déterminer le triplet (p,r,q) et donc le modèle qui correspond le mieux à notre problème. ARIMA est un algorithme adapté aux séries temporelles ce qui justifie notre choix d'application de cet algorithme.

2.4 Comparaison des algorithmes

Nous allons comparer les méthodes suivantes :

- Méthodes de Machine Learning:
Neural Networks - LSTM, Support vector machine (SVM)

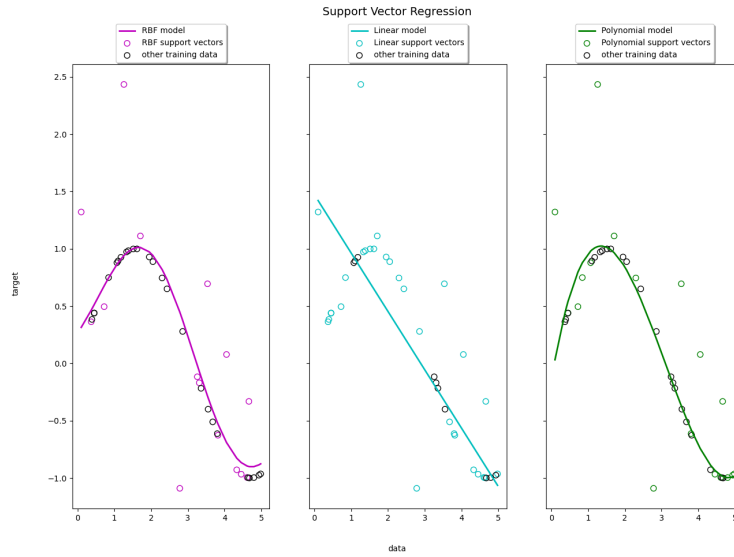


Figure 3: SVR with linear/non linear kernels

- Méthodes Statistiques:
ARIMA

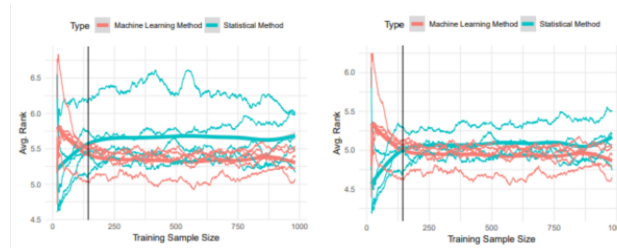


Figure 4

Les résultats représentés sur la Figure 4 montrent une tendance claire: Lorsque le nombre d'observations disponibles est petit, les méthodes statistiques présentent de meilleures performances. Cependant, à mesure que la taille de l'échantillon augmente, les méthodes d'apprentissage automatique les surpassent.

3 Démos et Résultats

3.1 LSTM

On récupère plusieurs scénarios des variations du débit sur les deux voies. On trace les données en vue d'en tirer des informations(Figure 5, 6). On remarque que le débits à une grande ne parrait pas stationnaire. Comme on le voit sur la Figure 9.

En principe, il n'est pas nécessaire de vérifier la stationnarité ni de la corriger lorsqu'on utilise un LSTM. Cependant, si les données sont stationnaires, cela contribuera à améliorer les per-

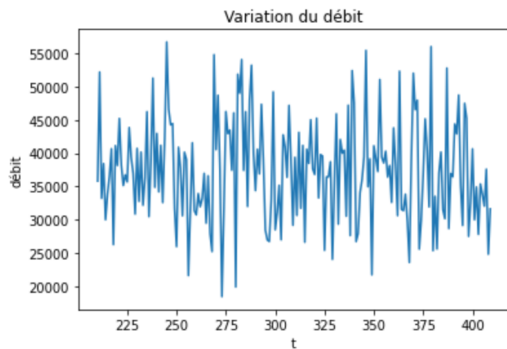


Figure 5

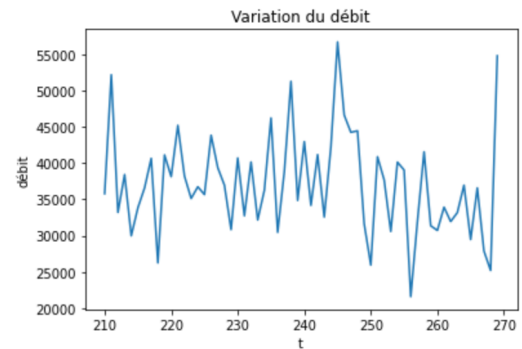


Figure 6

formances et facilitera l'apprentissage. On applique le log afin d'atténuer la variance. Et on va travailler sur la moyenne mobile de taille 7. Et obtient le plot sur la Figure 8 Code : lstm.ipynb

3.1.1 Implémentation

La taille de la couche d'entrée est le nombre des "lags" utilisés, ici 5. Une couche LSTM est une Dense d'une sortie. Eventuellement la valeur à prédire. On entraîne ainsi le réseau sur 70% des données et on obtient les résultats sur la Figure 10 11.

3.1.2 Echelle de prédiction

- Le mieux qu'on a pu faire est avec fenêtre de taille 7, Figure 12.
- On peut étendre sur l'horizon du temps (nombre de valeurs précédentes). Cependant, la taille des données d'entraînement croît linéairement.

3.1.3 Remarques

- + Les résultats sont pas mal.
- + Extension possible si le problème devient multivariés.
- Coût des transformations.

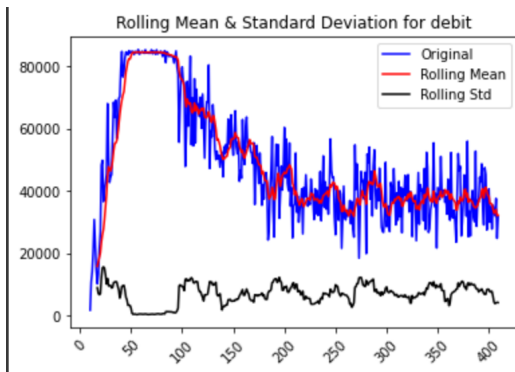


Figure 7: Rolling Mean and Standard Deviation pour le débit

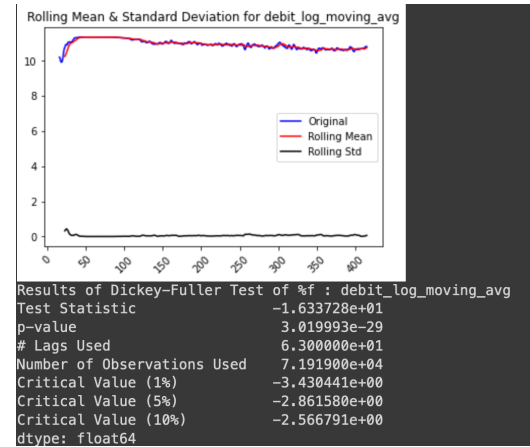


Figure 8: LogMovingAverage

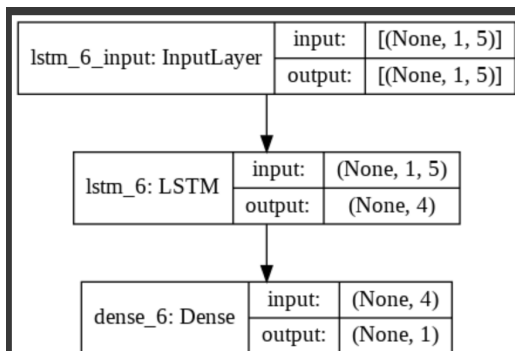


Figure 9: Ensemble des couches

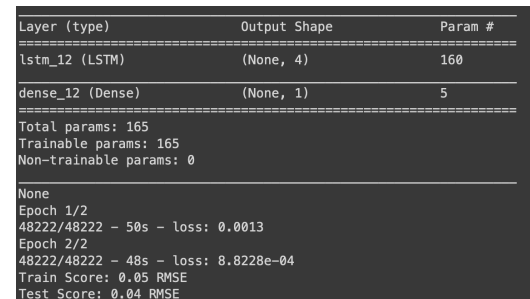


Figure 10: Metric: RMSE

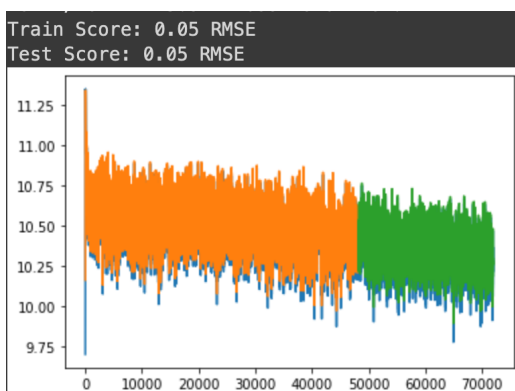


Figure 11: Fenêtre de taille 7

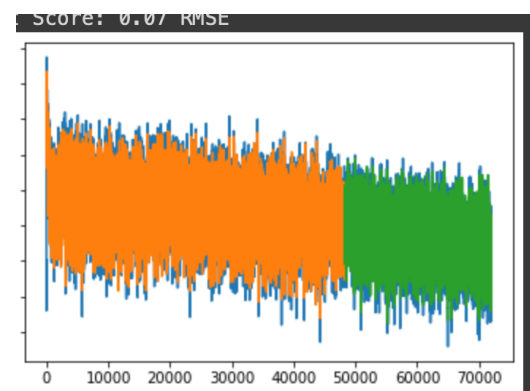


Figure 12: Fenêtre de taille 5

3.2 ARIMA

L'implémentation de cet algorithme est basé sur des packages Python : pmdarima (Spécifique pour ARIMA) statsmodels.tsa.seasonal (Plus général). On découvre le modèle pertinent à l'aide de la fonction `auto_arima`. On essaie avec 3 scénario et on obtient les résultats suivants.

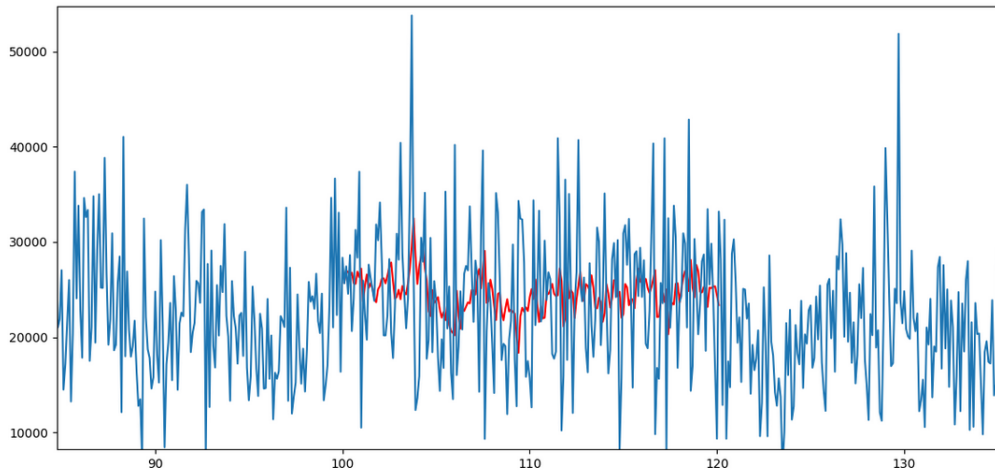


Figure 13: Scénario 1

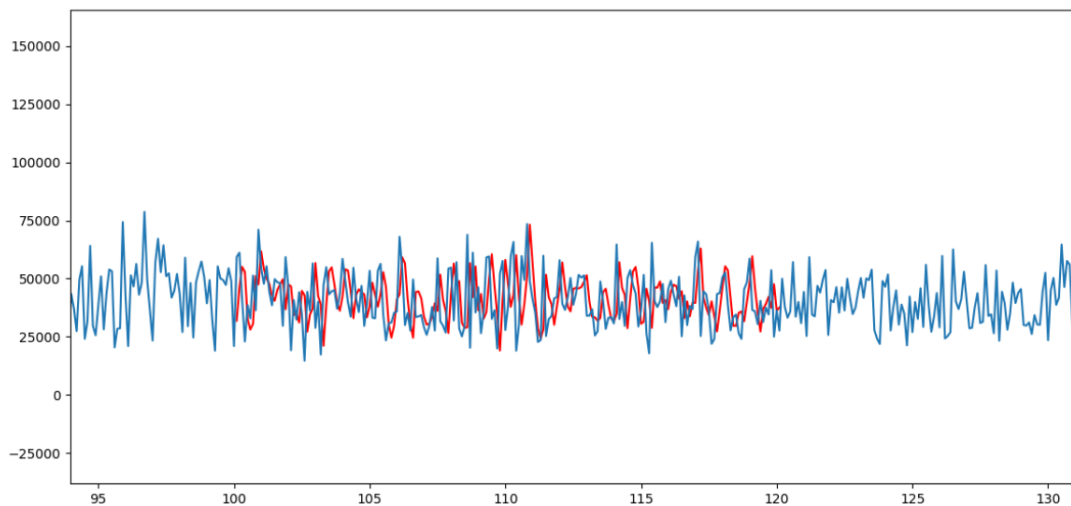


Figure 14: Scénario 2

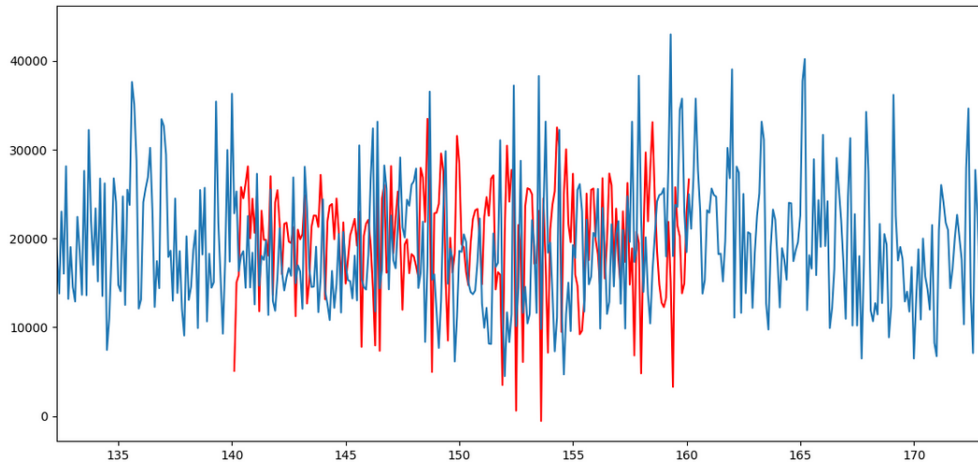


Figure 15: Scénario 3

3.2.1 Commentaires

(+) Ces prédictions commencent généralement après un temps d'apprentissage de 100 secondes (= 1,5 minutes) ce qui d'ARIMA un algorithme qui apprend "rapidement" et donc bien adapté aux problèmes dans lesquels on est limités en termes de données initiales. (+) On a obtenu des résultats satisfaisants quand il s'agit d'un débit stationnaire. En particulier pour le scénario 1, les résultats ont été de qualité inférieure aux résultats des scénarios 2 et 3 où les débit ont été stationnaires. (+) La qualité des résultats est plus ou moins indépendante du scénario : ARIMA offre une large de possibilité de modèles. (-)Par contre, puisque cet algorithme est basé sur le choix d'un modèle parmi ceux qui existent, on a besoin de tester tous les modèles et prendre celui qui minimise l'erreur. Ce processus peut prendre "beaucoup" de temps. Pratiquement on a remarqué une durée de 10 min pour les données proposées dans notre problème. ARIMA est donc un algorithme qui n'est pas adapté aux problèmes temps-réel.

3.3 SVM

3.3.1 RBF Kernel

Support Vector Regression avec un Kernel RadialBasisFunction. Figure 16 17. Erreur quadratique moyen RMSE=8101.97 (test score)

3.3.2 Polynomial Kernel

Erreur quadratique moyen RMSE=1785.45 (test score) Figure 18 19.

4 Conclusion

Il est bien possible de prédire le débit agrégée sur les deux voies aller et sortie du réseau satellite et les trois méthodes étudiées donnent des résultats acceptables. Comme les résultats

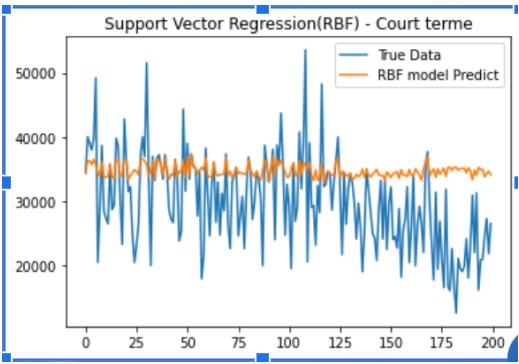


Figure 16: Court terme

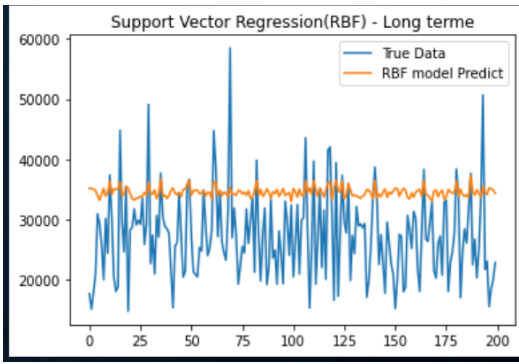


Figure 17: Long terme

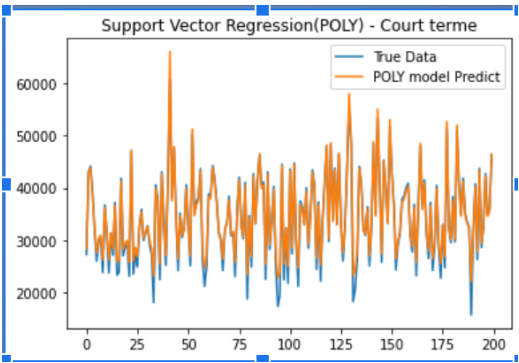


Figure 18: Court terme

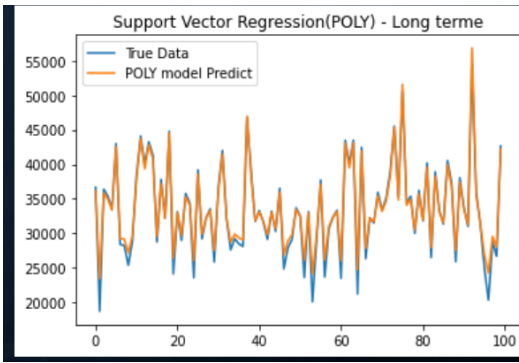


Figure 19: Long terme

le montrent, ARIMA est moins fiable que LSTM et SVM. Néanmoins, puisque SVM prend beaucoup plus de temps que LSTM, on en conclut que ce dernier est l'algorithme le plus adéquat pour faire la prédiction de trafic souhaitée.

References

- [1] Hochreiter. Untersuchungen zu dynamischen neuronalen netzen. URL <https://people.idsia.ch/~juergen/SeppHochreiter1991ThesisAdvisorSchmidhuber.pdf>.
- [2] Bongior. Learning long-term dependencies with gradient descent is difficult. URL <https://ieeexplore.ieee.org/document/279181>.